Configuring Your Experiment

Autobook

Everything you train and evaluate is controlled by a single YAML config file. This document explains what each field does and how to set up your config.

1. Your Config File

Each YAML file defines one experiment: the model to train, the GPU to use, and what to evaluate afterward. You write the config, run one command, and the pipeline handles the rest.

Place your config at `a3/<part>/configs/<name>.yaml`.

Convention: `p2_baseline.yaml`, `p2_ablation1.yaml`, `p4_final.yaml`.

See the templates at `a3/p2/configs/p2_template.yaml` and `a3/p4/configs/p4_template.yaml`.

2. General Structure

```yaml
experiment_name: <string>          # W&B group name
nanochat_ref: <branch-or-tag>      # git ref on the nanochat fork
gpu: <gpu-type>                    # Modal GPU type
timeout_hours: <number>            # max wall time per stage (hours)

train:                             # list of training stages
  - name: <string>                 # stage identifier
    depends_on: <stage-name>       # advanced, optional
    extra_iterations: <number>     # advanced, optional
    args:                          # nanochat CLI arguments
      depth: <number>              # model depth
      max_seq_len: <number>        # context length in tokens
      model_tag: <string>          # checkpoint folder name
      device_batch_size: <number>  # sequences per forward pass
      run: <string>                # W&B run name
      num_iterations: <number>     # optional, auto-computed
      save_every: <number>         # optional, checkpoint interval
      eval_every: <number>         # optional, val BPB interval
      log_every: <number>          # optional, W&B log interval

eval:                              # list of post-training evals
  - checkpoint: <model-tag>        # must match a model_tag above
    after_stage: <stage-name>      # which stage's checkpoint to eval
    evals: [bpb, core]             # built-in evals to run
    custom_eval: <path>            # optional, your eval script
```

The **`train:`** list can have one or more stages. Each stage runs on its own GPU container. The **`eval:`** list runs after all training completes. YAML keys in **`args`** use underscores; the runner converts them to CLI flags automatically (e.g. `max_seq_len` becomes `--max-seq-len`).

Multiple stages and evals. If you list multiple stages under **`train:`**, independent stages run in parallel on separate GPUs. Each eval entry under **`eval:`** targets one checkpoint (identified by **`checkpoint`** + **`after_stage`**), so add one entry per checkpoint you want to evaluate. All evals run in parallel after training finishes. For P2, you typically have one stage and one eval per YAML file (one file per ablation). For P4, one stage and one eval in a single file.

3. Field Reference

`experiment_name` — Any string. Becomes the W&B group name and appears in pipeline logs.

`nanochat_ref` — Branch or tag on the nanochat fork to check out. Use `baseline-v0` for unmodified nanochat, or your own branch name for modified code (see the *Getting Started* doc).

`gpu` — Modal GPU type. Options: `A100-80GB`, `A10G`, `H100`. Default: `A100-80GB`.

`timeout_hours` — Maximum wall time (in hours) per training or eval stage. Increase for larger models (depth 20 may need 6). Default: `3`.

`train:` — List of training stages. Each stage has:

- **`name`** — Any string. Referenced by **`after_stage`** in eval entries and by **`depends_on`** in later stages.
- **`depends_on`** — *(Advanced, optional.)* Name of a prior stage to resume from. The runner auto-computes the resume checkpoint. P2 and P4 don't need this.
- **`extra_iterations`** — *(Advanced, optional.)* Used with **`depends_on`**. Number of additional training steps beyond the prior

stage's final step.

args — Dict of nanochat training arguments. Fields:

depth — Model depth. 6 = picochat (∼136M params), 20 = full nanochat. Default: `20`.

max_seq_len — Context length in tokens. Default: `2048`.

model_tag — Checkpoint folder name. Pick something unique per experiment (e.g. `pico-baseline`, `d20-final`). Must match the **checkpoint** field in your eval entry. Default: `d{depth}`.

device_batch_size — Sequences per forward pass. If you get OOM errors, halve this value (e.g. 512 → 256 → 128). This does not change training dynamics — it only affects how many gradient accumulation steps are used. Default: `32`.

num_iterations — Total training steps. Omit to let nanochat auto-compute from scaling law. Default: auto-computed.

run — W&B run name. Use `"dummy"` to disable W&B logging. Pick something unique per run. Default: `"dummy"`.

save_every — Checkpoint save interval in steps. `-1` saves only at the end. Default: `-1`.

eval_every — Validation BPB evaluation interval during training. `-1` disables. Default: `-1`.

log_every — W&B logging interval in steps. `-1` disables. Default: `-1`.

core_metric_every — CORE benchmark interval during training. `-1` disables (run post-hoc via eval instead). Default: `-1`.

sample_every — Text generation sampling interval during training. `-1` disables. Default: `-1`.

eval: — List of evaluation entries. Each entry has:

checkpoint — Must match a **model_tag** from one of your training stages.

after_stage — Stage name whose final checkpoint to evaluate. The runner auto-resolves the step number. Alternatively, use **step: <number>** for an explicit step.

evals — List of evals to run. Options: `bpb`, `core`, `custom`. See Section 4 for what each does.

custom_eval — Path to your custom eval script, relative to `a3/`. Required when **evals** includes `custom`.

4. Evaluation

4.1 Custom eval

A custom eval is a Python script you write that loads a checkpoint and computes whatever metrics you need. Place it at `a3/<part>/evals/<name>.py`. The script must define a **run_eval** function:

```
def run_eval(checkpoint_dir: str, model_tag: str, step: int) -> dict:
    """Return a dict of results. Saved as JSON on the Volume."""
    return {"score": 0.42}
```

Reference it in your YAML by adding `custom` to the **evals** list and setting the **custom_eval** path:

```
eval:
  - checkpoint: pico-baseline
    after_stage: train
    evals: [bpb, core, custom]
    custom_eval: p3/evals/context_length_eval.py
```

The return dict is automatically saved as JSON on the Modal Volume.

4.2 Built-in evals

Two built-in evals are available:

bpb — Bits per byte on the training and validation splits. Always safe to run.

core — 22-task benchmark (HellaSwag, ARC, PIQA, BoolQ, etc.). Only use if the model was trained with **max_seq_len: 2048** or higher — shorter contexts may crash because CORE prompts exceed the RoPE cache.

Use them in the **evals** list:

```
eval:
  - checkpoint: pico-baseline
    after_stage: train
    evals: [bpb, core]
```